

GYM TRACKER

Complete Project Report & Demonstration Record

Command-Line Fitness Tracking Application

Prepared by: **Yogesh Kumar Singh**

Technology: Python 3 · Standard Library · JSON Storage

Project type: Modular terminal-based application

Purpose of this report: document the problem, requirements, design diagrams, implementation structure, pseudocode, observed terminal outputs, testing evidence, challenges, learnings, and future enhancements for the Gym Tracker project.

Submission-ready report — 19 pages

1. Executive Summary

Gym Tracker is a command-line Python application designed to record strength-training sessions. A user can create Push, Pull, or Legs workouts, enter exercises with sets, repetitions and weight, review workout history, calculate total training volume, search for exercises, and view summary statistics. Workout data is stored locally in JSON so that previous sessions remain available after the program is closed.

The current terminal demonstration confirms the main user-facing workflow. The screenshots supplied with this report show successful workout creation, history retrieval, total-volume calculation, exercise search, workout statistics, and a clean exit from the application.

The project is organized into meaningful modules rather than keeping every responsibility in one file. The documented structure separates the exercise model, workout model, analytics, storage, tests, and main application workflow. This supports the project guide's expectations for modular implementation, validation, testing, documentation, and version control.

Contents

Page	Section
3	Introduction, problem statement and objectives
4	Functional requirements
5	Non-functional requirements and technical expectations
6	System architecture
7	Workflow and pseudocode
8	Use Case Diagram
9	Sequence Diagram
10	Class / Component Diagram
11	ER / JSON Storage Design
12	Implementation and module-by-module explanation
13–17	Screenshots and observed outputs
18	Testing and output verification
19	Challenges, learnings, future enhancements and references

2. Introduction, Problem Statement & Objectives

2.1 Introduction

Keeping track of gym training manually can become inconvenient when a person wants to remember exactly what they lifted, how many sets they performed, or how their total training volume changes over time. Gym Tracker addresses this with a small offline application that focuses on the core information needed for strength-training records.

The application deliberately uses a terminal interface instead of a graphical interface. This keeps the project closely aligned with introductory Python concepts such as input/output, conditionals, loops, functions, classes, collections, file handling, JSON, validation, and modular programming.

2.2 Problem Statement

It is easy to forget exactly what was lifted from one gym session to the next. Gym Tracker provides a simple, offline place to record exercises, sets, repetitions and weight, then retrieve and summarise that information later.

2.3 Objectives

Objective	Description
Record workouts	Store Push, Pull and Legs sessions with a date.
Record exercises	Capture exercise name, sets, repetitions and weight.
Persist data	Save workout information locally in JSON.
Retrieve history	Display previously stored sessions and their volume.
Analyse training	Calculate total volume and workout-type statistics.
Search records	Find an exercise across saved workout history.
Validate input	Reject invalid workout categories and invalid numeric values.
Demonstrate Python concepts	Use classes, functions, collections, loops, conditionals and file handling in one practical project.

3. Functional Requirements

The project guide requires at least three major functional modules, clear input/output, and a logical user workflow. Gym Tracker satisfies this through workout management, history/data retrieval, and analytics/search.

Module	Function	Input	Output / Result
Workout Management	Create a workout and add exercises	Workout type, date, exercise name, sets, reps, weight	Workout object is created and saved
History	View saved workouts	Menu option 2	Date, workout type, exercises and session volume
Volume Analytics	Calculate total training volume	Saved workout records	Total volume in kg
Exercise Search	Find an exercise in history	Exercise name	Matching workout records
Workout Statistics	Summarise records	Saved workouts	Workout count, exercise count, total volume, Push/Pull/Legs counts
Persistence	Save and load data	Workout objects / JSON	Data survives program restarts
Validation	Check user input	Menu choices, workout type, numbers	Valid records or an error/retry path

Major functional modules

1. Workout Management: the central data-entry workflow. **2. History & Search:** lets the user inspect previously stored records. **3. Analytics:** calculates volume and workout statistics. Storage and validation support these modules without being separate user-facing projects.

Clear input/output structure

The screenshots demonstrate the input/output flow directly: the user chooses a menu option, enters values, and receives a formatted result. After adding a workout, the application returns to the main menu rather than terminating.

4. Non-Functional Requirements & Technical Expectations

Requirement	How Gym Tracker addresses it
Usability	Simple numbered terminal menu and direct prompts make the workflow easy to follow.
Reliability	Workout records are persisted to a local JSON file and loaded again when the application starts.
Maintainability	Responsibilities are split into modules such as exercise, workout, analytics, storage and main.
Error handling	Numeric values and workout categories are validated; invalid entries can be rejected instead of being silently stored.
Resource efficiency	The project uses Python's standard library and a small local JSON file rather than a heavy external database.
Portability	The application is command-line based and uses Python standard-library features.
Scalability	The modular structure leaves room for additional analytics, data fields, or storage methods.

Technical expectations addressed

Expectation	Project evidence
Architectural design	Module-level architecture and workflow diagrams in this report.
Subject concepts	Input/output, conditionals, loops, functions, classes, lists, dictionaries, file handling and JSON.
Modular implementation	Separate files for exercise, workout, analytics, storage, main and tests.
Documentation	README, project statement and this report.
Validation	Input checks for workout categories and numeric exercise values.
Testing	Automated tests are included in the project structure; manual output verification is documented here.
Version control	Git/GitHub is part of the project submission structure and README requirements.

The project guide specifies these technical and documentation expectations but does not prescribe a minimum or maximum report page count. This report intentionally exceeds 15 pages so that the required artefacts and the actual demonstration evidence can be shown clearly.

5. System Architecture

The application follows a simple modular architecture. The main program coordinates the user interaction. Exercise and Workout represent the core data. Analytics performs calculations and searches. Storage handles JSON persistence. Tests verify important behaviour.

Layer / File	Responsibility
main.py	Runs the application loop, displays the menu, receives input and calls the appropriate project functionality.
exercise.py	Defines the exercise data model and exercise-related behaviour/validation.
workout.py	Defines a workout and manages its collection of exercises and workout-level calculations.
analytics.py	Provides total volume, exercise search and workout statistics.
storage.py	Converts workout data to/from JSON and handles local persistence.
test_gym_tracker.py	Contains automated checks for important application behaviour.
workouts.json	Local persistent data file containing saved workout records.

Architecture flow: User → main.py → Exercise / Workout objects → Analytics and Storage → workouts.json. Results then return through main.py to the terminal.

Why this architecture is useful

Separating responsibilities means that a change to JSON storage does not require rewriting the menu, and an analytics calculation can be tested independently from the input prompts. It also makes the project easier to explain during a viva because each file has a clear purpose.

6. Process Flow / Workflow & Pseudocode

The normal application workflow is a repeating menu loop. The user selects one of six actions. After completing an action, the program returns to the menu. Choosing option 6 ends the loop.

6.1 Main-menu pseudocode

```
START
LOAD saved workout data

WHILE the user has not chosen Quit:
DISPLAY the Gym Tracker menu
READ menu choice

IF choice = 1:
ADD a workout
ASK for workout type and date
REPEAT:
ASK for exercise name, sets, reps and weight
ADD exercise to workout
ASK whether another exercise is needed
SAVE workout data
DISPLAY confirmation

ELSE IF choice = 2:
DISPLAY workout history

ELSE IF choice = 3:
CALCULATE and DISPLAY total training volume

ELSE IF choice = 4:
ASK for exercise name
SEARCH saved workouts
DISPLAY matching records

ELSE IF choice = 5:
CALCULATE and DISPLAY workout statistics

ELSE IF choice = 6:
DISPLAY exit message
STOP

ELSE:
DISPLAY invalid-option message
END
```

6.2 Volume-calculation pseudocode

```
FOR each workout:
FOR each exercise:
exercise_volume = sets × reps × weight
add exercise_volume to workout volume
add workout volume to total volume

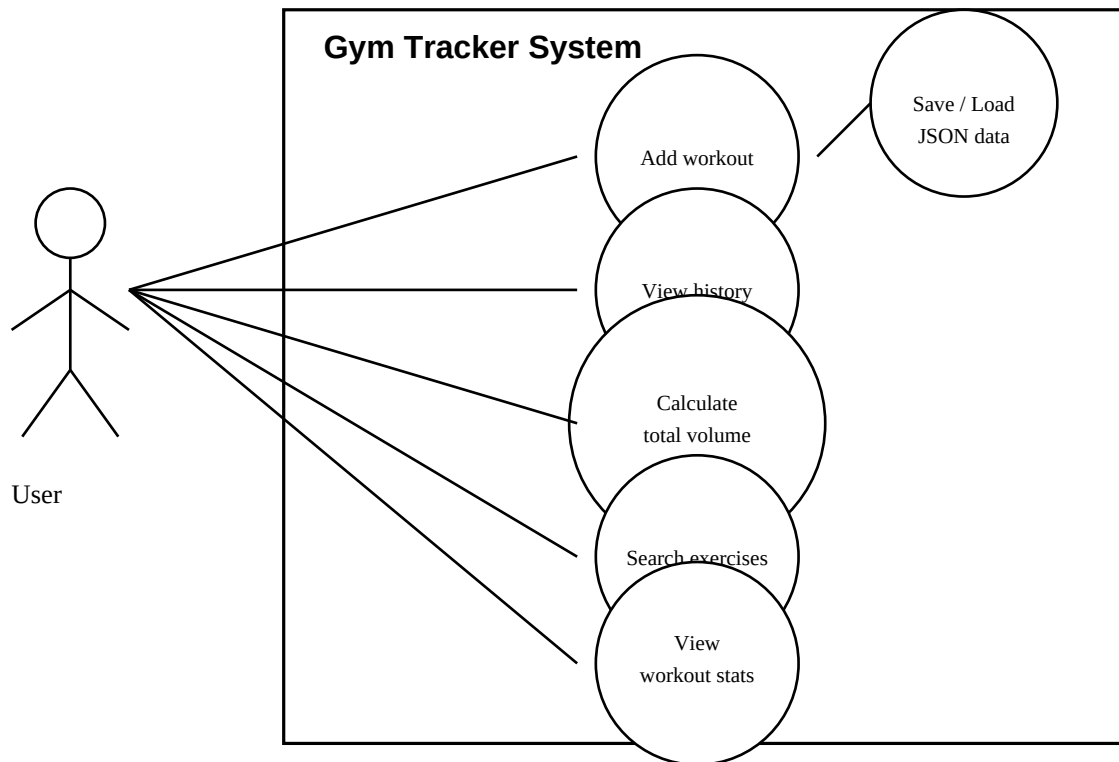
DISPLAY total volume
```

6.3 Search pseudocode

```
READ the exercise name to search for
FOR each workout:
FOR each exercise:
IF searched name matches exercise name:
DISPLAY the workout and matching exercise
IF no match was found:
DISPLAY that the exercise was not found
```

7. Use Case Diagram

The main actor is the User. The user interacts with the application through the terminal menu. Saving and loading JSON data are supporting system actions rather than separate applications.

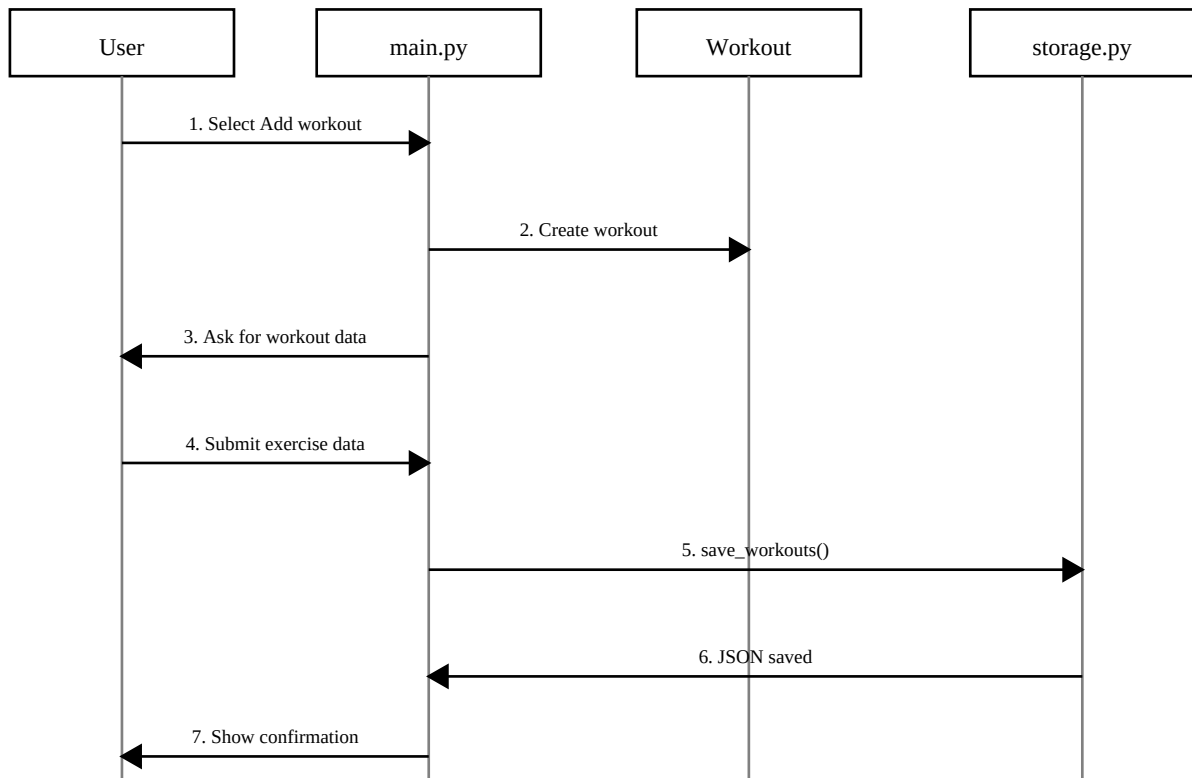


Use-case explanation

Use Case	Description
Add workout	Create a dated Push/Pull/Legs session and enter one or more exercises.
View history	Display all saved workouts and their session volume.
Calculate total volume	Aggregate the volume of all stored exercises.
Find an exercise	Search the saved history by exercise name.
View workout stats	Show counts of workouts/exercises and workout-type totals.
Save / Load JSON	Persist records between application runs.

8. Sequence Diagram

The sequence below represents the successful 'Add a workout' flow observed in the terminal demonstration. The user interacts with `main.py`; the application builds the workout and exercise data, then sends the completed data to the storage layer.



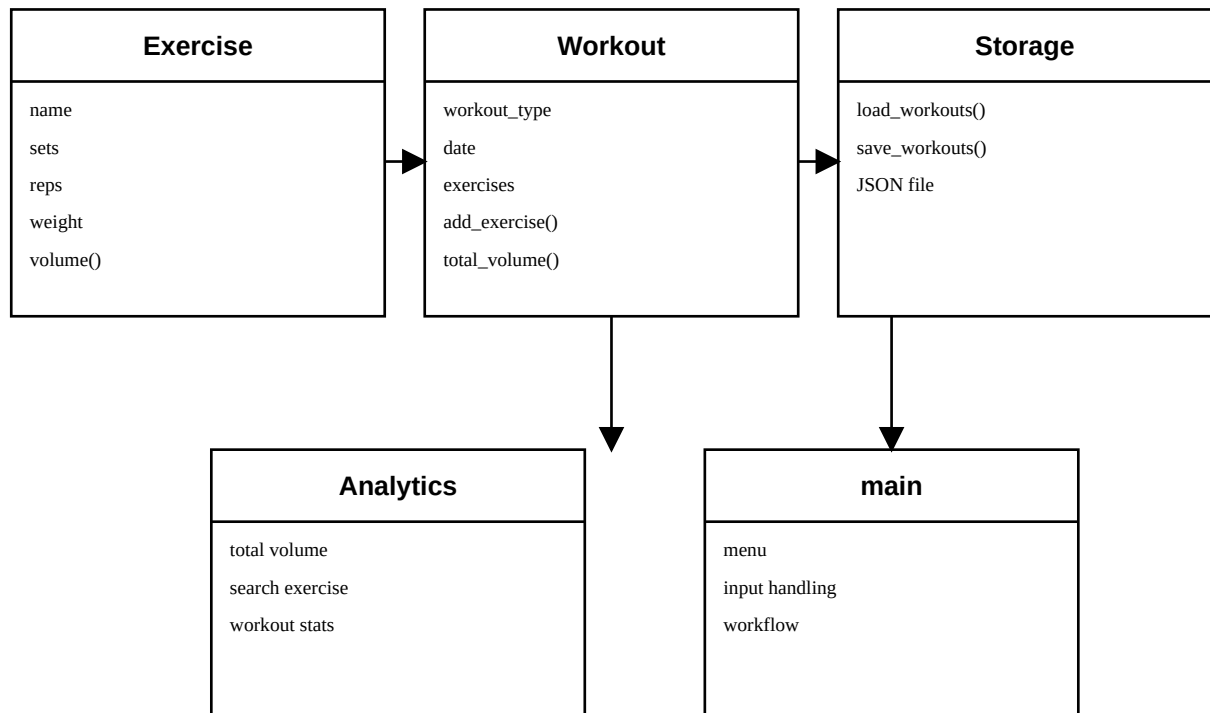
Observed sequence in the supplied run

The user selected option 1, selected Push, entered the date 29/09/2026, entered Bench Press and Incline Dumbbell Press, supplied sets/reps/weight, answered YES to add another exercise, then answered no. The application printed **“Workout saved. Nice work!”** and returned to the menu.

Important: this diagram is a design representation of the observed workflow. It is not a claim that every internal function call has exactly these names or timings; the terminal output is the primary evidence for the user-visible sequence.

9. Class / Component Diagram

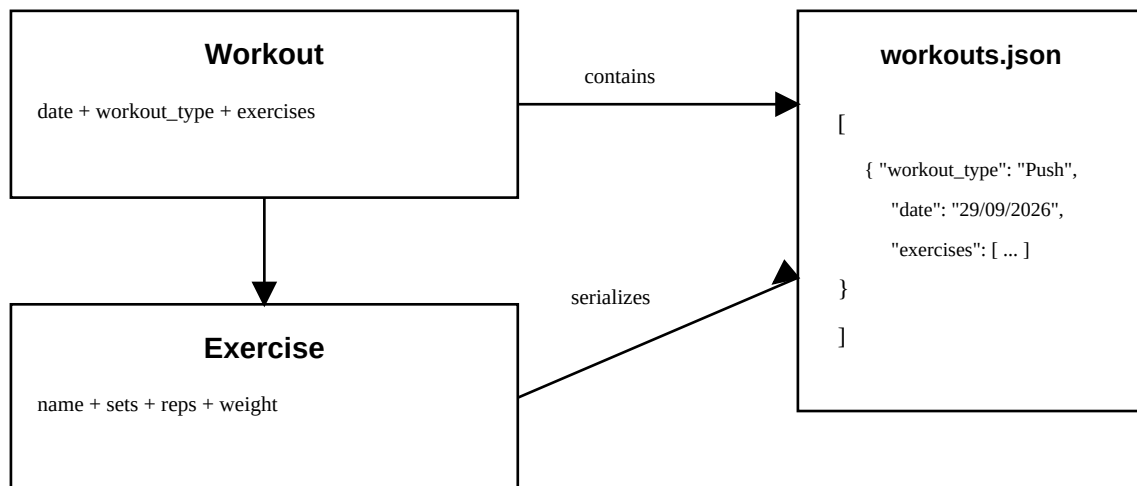
The project separates the core data models from the supporting analytics and persistence components. The exact public methods may evolve during development, but the responsibilities shown here match the current modular project structure and documented behaviour.



Component	Role in the project
Exercise	Stores exercise-level values: name, sets, reps and weight; supports volume calculation.
Workout	Stores workout type/date and a collection of Exercise records; supports adding exercises and workout-level volume calculation.
Analytics	Calculates totals, searches exercises and produces workout statistics.
Storage	Loads and saves workout records using JSON.
main	Coordinates the terminal interface and calls the other components.

10. ER / JSON Storage Design

Because the project uses a JSON file rather than a relational database, the diagram is best understood as an entity/storage relationship model. It shows the same conceptual relationship the project needs: a Workout contains one or more Exercise records, and those records are serialized into workouts.json.



Conceptual structure

Workout

- workout_type
- date
- exercises[]

Exercise

- name
- sets
- reps
- weight

JSON representation:

```
[  
{  
  "workout_type": "Push",  
  "date": "29/09/2026",  
  "exercises": [  
    {  
      "name": "Bench Press",  
      "sets": 3,  
      "reps": 12,  
      "weight": 100  
    }  
  ]  
}
```

Storage behaviour

When a workout is added, its information is converted into JSON-compatible data and written to the local file. When the application starts, the saved data can be loaded back into the application's workout/exercise structures. The history screenshot provides practical evidence that earlier records remained available across runs.

11. Implementation Details — Module by Module

File	What the code is responsible for	User-visible output?
main.py	Menu, input collection, validation flow, dispatching options, confirmation and exit.	YES — directly drives all five feature outputs and exit.
exercise.py	Exercise data structure and exercise-level calculations/validation.	NO standalone menu; output appears through main.py.
workout.py	Workout data structure, exercise collection and workout volume.	NO standalone menu; output appears through main.py.
analytics.py	Total volume, search and workout statistics logic.	NO standalone menu; functions are called by main.py.
storage.py	JSON save/load functionality.	NO standalone menu; effect is visible indirectly through saved history.
test_gym_tracker.py	Automated verification of project behaviour.	YES when tests are run; no test-run screenshot is included here.
workouts.json	Persistent data file.	Not a program; its effect is visible when history reloads saved records.

Example:

```
3 sets × 12 reps × 100 kg
= 3600 kg
```

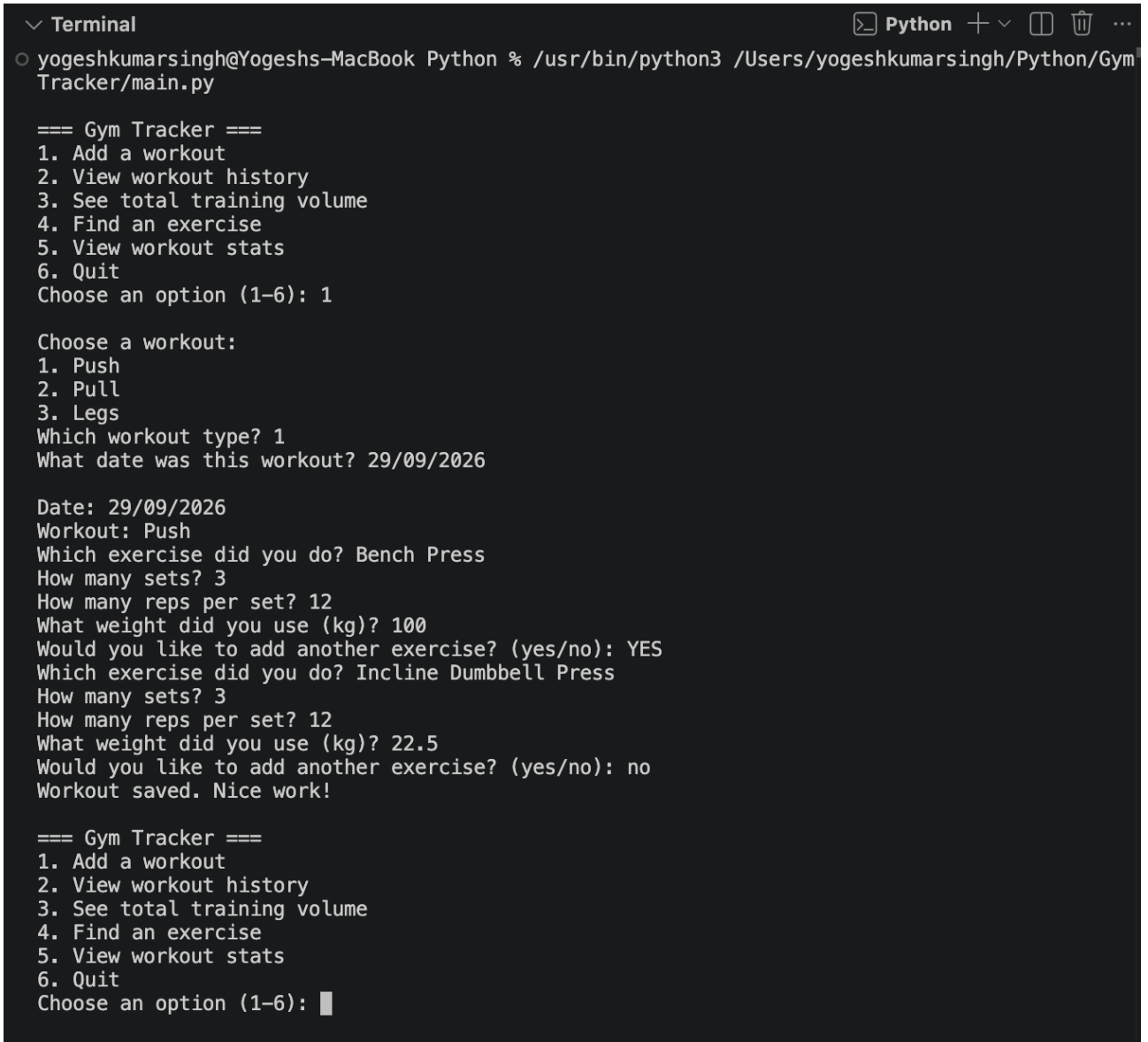
For the 29/09/2026 Push workout shown in the screenshot:

```
Bench Press: 3 × 12 × 100 = 3600 kg
Incline Dumbbell Press: 3 × 12 × 22.5 = 810 kg
-----
Workout total: 4410 kg
```

This exact 4410.0 kg result appears in the workout-history screenshot, providing a direct consistency check between the input values and the program's calculation.

12. Demonstration Evidence A — Adding a Workout

The first supplied screenshot shows the complete add-workout interaction. The user selects option 1, chooses Push, enters the date 29/09/2026, then records two exercises.



```
Terminal
yogeshkumarsingh@Yogeshs-MacBook Python % /usr/bin/python3 /Users/yogeshkumarsingh/Python/GymTracker/main.py

=== Gym Tracker ===
1. Add a workout
2. View workout history
3. See total training volume
4. Find an exercise
5. View workout stats
6. Quit
Choose an option (1-6): 1

Choose a workout:
1. Push
2. Pull
3. Legs
Which workout type? 1
What date was this workout? 29/09/2026

Date: 29/09/2026
Workout: Push
Which exercise did you do? Bench Press
How many sets? 3
How many reps per set? 12
What weight did you use (kg)? 100
Would you like to add another exercise? (yes/no): YES
Which exercise did you do? Incline Dumbbell Press
How many sets? 3
How many reps per set? 12
What weight did you use (kg)? 22.5
Would you like to add another exercise? (yes/no): no
Workout saved. Nice work!

=== Gym Tracker ===
1. Add a workout
2. View workout history
3. See total training volume
4. Find an exercise
5. View workout stats
6. Quit
Choose an option (1-6): █
```

Figure 1. Terminal output: successful creation and saving of a Push workout with Bench Press and Incline Dumbbell Press.

Observed result: **PASS**. The program accepted the workout details, accepted a second exercise after the YES response, printed “Workout saved. Nice work!”, and returned to the main menu.

13. Demonstration Evidence B — Workout History

The second supplied screenshot shows option 2 and the resulting saved workout history. It displays three stored workout records and calculates a total volume for each session.

```
=== Gym Tracker ===
1. Add a workout
2. View workout history
3. See total training volume
4. Find an exercise
5. View workout stats
6. Quit
Choose an option (1-6): 2

Your workout history

Date: 29/09/2026
Workout: Push
Bench Press: 3 sets * 12 reps * 100 kg
Incline Dumbbell Press: 3 sets * 12 reps * 22.5 kg
Total Volume: 4410.0 kg

Date: 28/08/2026
Workout: Push
Bench Press: 3 sets * 12 reps * 100 kg
Incline Dumbbell Press: 3 sets * 12 reps * 80 kg
Total Volume: 6480.0 kg

Date: 29/09/2026
Workout: Push
Bench Press: 3 sets * 12 reps * 100 kg
Incline Dumbbell Press: 3 sets * 12 reps * 22.5 kg
Total Volume: 4410.0 kg

=== Gym Tracker ===
1. Add a workout
2. View workout history
3. See total training volume
4. Find an exercise
5. View workout stats
6. Quit
Choose an option (1-6): █
```

Figure 2. Terminal output: workout history and per-workout volume.

The displayed values can be independently checked: $3600 + 810 = 4410$ kg for the 29/09/2026 entry; $3600 + 2880 = 6480$ kg for the 28/08/2026 entry; and the later 29/09/2026 entry is another 4410 kg.

14. Demonstration Evidence C — Total Training Volume

The third supplied screenshot shows option 3. The application reports a total workout volume of 15300.0 kg.

```
=== Gym Tracker ===
1. Add a workout
2. View workout history
3. See total training volume
4. Find an exercise
5. View workout stats
6. Quit
Choose an option (1-6): 3
Total workout volume: 15300.0 kg

=== Gym Tracker ===
1. Add a workout
2. View workout history
3. See total training volume
4. Find an exercise
5. View workout stats
6. Quit
Choose an option (1-6): █
```

Figure 3. Terminal output: total training volume.

Cross-check: $4410 + 6480 + 4410 = \mathbf{15300\ kg}$. The program's result therefore matches the visible workout-history records.

Verification status: PASS. The total-volume output is consistent with the session-level totals shown in the previous screenshot.

15. Demonstration Evidence D — Exercise Search

The fourth supplied screenshot shows option 4 with the search term “Bench Press”. The application returns every stored workout containing that exercise.

```
=== Gym Tracker ===
1. Add a workout
2. View workout history
3. See total training volume
4. Find an exercise
5. View workout stats
6. Quit
Choose an option (1-6): 4
Which exercise are you looking for? Bench Press

Date: 29/09/2026
Workout: Push
Bench Press: 3 sets * 12 reps * 100 kg

Date: 28/08/2026
Workout: Push
Bench Press: 3 sets * 12 reps * 100 kg

Date: 29/09/2026
Workout: Push
Bench Press: 3 sets * 12 reps * 100 kg

=== Gym Tracker ===
1. Add a workout
2. View workout history
3. See total training volume
4. Find an exercise
5. View workout stats
6. Quit
Choose an option (1-6): █
```

Figure 4. Terminal output: exercise search results.

The screenshot returns three matching records, corresponding to the three stored workouts. This demonstrates that the search logic traverses workout history rather than only checking the most recently added session.

16. Demonstration Evidence E — Workout Statistics & Exit

The fifth supplied screenshot contains the workout-statistics output followed by a normal exit using option 6.

```
=== Gym Tracker ===
1. Add a workout
2. View workout history
3. See total training volume
4. Find an exercise
5. View workout stats
6. Quit
Choose an option (1-6): 5

Your workout stats
Total workouts: 3
Total exercises: 6
Total volume: 15300.0 kg
Push workouts: 3
Pull workouts: 0
Legs workouts: 0

=== Gym Tracker ===
1. Add a workout
2. View workout history
3. See total training volume
4. Find an exercise
5. View workout stats
6. Quit
Choose an option (1-6): 6
See you next workout!
yogeshkumarsingh@Yogeshs-MacBook Python %
```

Figure 5. Terminal output: workout statistics and successful program exit.

Observed statistics: 3 total workouts, 6 total exercises, 15300.0 kg total volume, 3 Push workouts, 0 Pull workouts and 0 Legs workouts. The program then prints “See you next workout!” and returns to the shell prompt.

The final shell prompt is important evidence: unlike the earlier Ctrl+C traces, this run ended through the application's own Quit option rather than a KeyboardInterrupt.

17. Testing, Output Verification & Evidence Matrix

The table below separates what has been directly demonstrated in the supplied terminal evidence from functionality that is internal/supporting and therefore does not normally print its own menu.

Feature / Code	Evidence	Status
main.py — Add workout	Screenshot 1: workout accepted and saved	PASS
main.py — View history	Screenshot 2: saved records displayed	PASS
main.py — Total volume	Screenshot 3: 15300.0 kg	PASS
main.py — Search	Screenshot 4: Bench Press found in three records	PASS
main.py — Statistics	Screenshot 5: counts and total volume	PASS
main.py — Quit	Screenshot 5: normal shell prompt after Quit	PASS
Workout / Exercise logic	Results are reflected in history, volume and statistics	PASS (indirect)
Analytics module	Functions are reflected in total volume/search/stats; standalone module is not the app entry point.	PASS (indirect)
Storage module	Prior records are available in history after restart	PASS (indirect)
Input validation	Project contains validation logic; these five screenshots do not demonstrate an invalid-input case.	NOT CAPTURED HERE
Automated test suite	Project includes test_gym_tracker.py; the five screenshots do not show a test-run terminal.	NOT CAPTURED HERE

Manual consistency checks

Check	Calculation / observation	Result
29/09 session volume	$3600 + 810 = 4410$	Matches screenshot
28/08 session volume	$3600 + 2880 = 6480$	Matches screenshot
Total volume	$4410 + 6480 + 4410 = 15300$	Matches screenshot
Exercise count	$2 + 2 + 2 = 6$	Matches statistics
Workout count	3 stored sessions	Matches statistics
Push count	All three records are Push	Matches statistics

Note on evidence scope

A feature marked 'NOT CAPTURED HERE' is not being called broken. It simply means the supplied screenshots do not prove that particular case. For a final submission, running the automated tests and one invalid-input example would make the evidence set even stronger.

18. Challenges, Learnings, Future Enhancements & References

18.1 Challenges Faced

Challenge	How it was handled
Keeping the project manageable	Responsibilities were separated into modules instead of one continuously growing file.
Input errors	Numeric and category validation were introduced so invalid values can be rejected.
Persistent data	JSON storage was added so workouts remain available between runs.
Search correctness	The search workflow was tested using an exercise appearing in multiple records.
Output verification	The same volume values were calculated manually and compared with program output.

18.2 Learnings & Key Takeaways

The project demonstrates that a small application can combine multiple Python concepts into one coherent workflow. The most important learning is not just writing individual statements, but connecting data structures loops, functions, classes and file handling so that information moves through the entire application. JSON persistence also shows how in-memory Python objects can be represented as stored data.

18.3 Future Enhancements

Enhancement	Purpose
Date validation	Ensure entered dates follow a consistent format and represent valid dates.
Duplicate-session handling	Warn the user when an identical workout/date is entered accidentally.
Progress tracking	Compare an exercise's weight or volume across dates.
Personal records	Identify highest recorded weight or volume for each exercise.
CSV export	Allow users to export records for spreadsheet analysis.
Graphical dashboard	Add charts for weekly/monthly volume in a later version.
More automated tests	Add explicit tests for invalid input and edge cases.
Improved search	Support partial matches, case-insensitive matching and filters by workout type/date.

18.4 References

1. VITyarthi — *Build Your Own Project: General Project Instructions & Submission Guidelines*, provided as the project guide for this submission.
2. Python Standard Library documentation concepts used by the project: JSON serialization, dataclasses, unittest and basic file handling.
3. Project source files and README/statement documentation maintained as part of the Gym Tracker project.

Final status: The supplied terminal evidence demonstrates the application's core end-to-end workflow: create → save → reload/history → calculate → search → analyse → quit. The report also includes the required design artefacts, pseudocode, screenshots, output verification and discussion of evidence gaps.